

SFA – Data Model and Product Catalog

Overview

SFA's data model is built around a single principle: **PostgreSQL is the single source of truth**. There are no intermediate files, no Excel exports, no ad-hoc JSON files that drive system behavior. Every alias, every scope-skip rule, every unit conversion, every product definition, every pipeline run record – it all lives in the database and is accessed via SQLAlchemy 2.x ORM.

The database has grown through **31 Alembic migrations** over the course of the project, evolving from the initial 23-table schema at M1 to the current 31-table schema at M9+. All schema changes are versioned, reversible, and documented.

Database Summary

Attribute	Value
Engine	PostgreSQL 15
ORM	SQLAlchemy 2.x
Migrations	Alembic (31 revisions, head: 031)
Environment	Docker (dev port 5433), production on waldhomeserver
Connection config	<code>DATABASE_URL</code> environment variable
Timezone	All timestamps: <code>TIMESTAMPTZ</code>
Financial precision	All price columns: <code>NUMERIC (12, 4)</code> - no FLOAT anywhere

Table Groups

The 31 tables are organized into five logical groups:

Group 1: Source Registry

data_sources - The registry of all farms, CSAs, farm shops, and farmers markets that SFA monitors.

Key columns:

- **source_code** - unique short identifier (e.g., `farm_lehem_geshem`)
- **display_name** - human-readable name in Hebrew
- **source_type** - `easyfarm_api` / `standalone_html` / `govt_benchmark`
- **collector_type** / **parser_type** - which collection and parsing implementation to use
- **is_active** - whether this source is currently being collected
- **fetch_schedule_cron** - cron expression for collection timing
- **fetch_profile_json** - HTTP headers, payload, auth tokens if needed

source_fetch_runs - A record of every fetch attempt: timestamp, outcome, HTTP status code, retry count, error details if failed.

raw_assets - The raw HTTP responses. Each row stores the full response body (or a reference to the file), the content checksum (for deduplication), fetch timestamp, and source ID. Checksum deduplication means re-fetching unchanged content produces no new rows.

Group 2: Raw Extracted Items

raw_extracted_items - The output of the parser stage. Each row is one product line extracted from a raw asset.

Key columns:

- **raw_name** - the product name exactly as it appeared on the source (Hebrew, unmodified)
- **raw_price_str** - the price string exactly as found ("12.50", "12-15", "15.00 ₪")
- **raw_unit** - the unit string as found ("אגודה", "יחידה", "ק"ג")
- **raw_qty_str** - the quantity string if present ("500 יחידות", "5", "גרם")
- **source_id** - foreign key to `data_sources`
- **raw_asset_id** - foreign key to `raw_assets`
- **status** - `extracted` → `normalized` / `unresolvable` / `ignored`
- **ignore_reason_code** - if ignored: `approved_scope_skip` or specific reason

The **status** field is the key lifecycle indicator. Newly parsed items start as `extracted`. The normalizer transitions them to `normalized`, `unresolvable`, or `ignored`. The

`full_data_refresh` CLI command resets all items to `extracted` and re-runs the pipeline from scratch.

Group 3: Normalization Catalog

This group contains the data that drives the normalization pipeline. These tables are what make SFA's normalization **data-driven** – changing a rule or alias requires no code changes, only a database update.

products – The canonical product catalog. 67 products, each with:

- `canonical_name` – the normalized product name (Hebrew)
- `display_name_he` – display name for the public index
- `product_category` – vegetable category grouping
- `unit_type` – `kg` / `unit` / `bunch` / `basket` / `pack`
- `is_basket_product` – true for CSA basket products (treated independently, not decomposed to per-kg)
- `is_active` – whether this product appears in the public index
- `display_order` – sort order in the published table

product_aliases – 232 mappings from raw Hebrew product names to canonical products.

Key columns:

- `raw_name` – the raw string to match (Hebrew, as it appears on source websites)
- `product_id` – foreign key to `products`
- `alias_type` – `exact` / `global` / `substring` (priority order in matching)
- `source_id` – optional: restricts this alias to a specific source (source-specific overrides)
- `is_active` – whether this alias is currently used

The three alias types form a priority hierarchy:

1. `exact` – exact string match (highest priority, used when a specific farm's exact phrasing is known)
2. `global` – normalized/cleaned string match (strip whitespace, normalize characters)
3. `substring` – partial match anywhere in the string (lowest priority, broadest coverage)

catalog_scope_skip_rules – 301 rules for filtering out non-food and out-of-scope items.

Key columns:

- `pattern` – the matching pattern (string)

- `rule_type` - `exact` / `prefix` / `contains` / `regex`
- `skip_category` - `grocery` / `dry_grocery` / `donation` / `cleaning` / `other`
- `is_active` - whether this rule is active
- `notes` - human-readable explanation of why this rule exists

When a raw item matches any active scope-skip rule, its status is set to `ignored` with `ignore_reason_code = approved_scope_skip`. The item is not deleted – it remains in the database for audit purposes.

`measurement_units` - 11 canonical measurement units: `kg`, `gram`, `unit`, `bunch`, `basket_small`, `basket_large`, `pack`, `liter`, `ml`, `dozen`, `box`.

`unit_conversions` - Conversion factors for non-kg units to their ₪/kg equivalents. Examples:

- `bunch` → 0.5 kg (half a kilogram per bunch, for most leafy products)
- `unit` (zucchini) → 0.35 kg
- `unit` (cabbage) → 1.2 kg
- `basket_small` → treated as independent product, no per-kg conversion

Conversions are **product-specific** - “one bunch of parsley” and “one bunch of celery” do not weigh the same. The `unit_conversions` table joins products to units with a specific `conversion_factor`.

Group 4: Normalized Observations and Aggregates

`normalized_observations` - The output of the normalizer. One row per successfully normalized product observation.

Key columns:

- `product_id` - canonical product (foreign key to `products`)
- `source_id` - which source contributed this observation
- `raw_extracted_item_id` - the source raw item (for full audit trail)
- `price_shekel_per_unit` - normalized price in ₪/kg equivalent (`NUMERIC(12,4)`)
- `measurement_unit_id` - the resolved canonical unit
- `quantity` - the resolved quantity
- `is_organic` - whether the organic certification flag was detected
- `confidence_score` - 0.0-1.0 (based on alias type and unit resolution quality)
- `observation_date` - date of the fetch

- `admin_flag` - `none` / `hide` / `review` (admin override without deletion)
- `status` - `normalized` / `superseded`

daily_aggregates - The aggregated daily statistics per product.

Key columns:

- `product_id` - canonical product
- `aggregate_date` - the date of the aggregate
- `avg_price` - mean ₪/kg (`NUMERIC(12,4)`)
- `median_price` - median ₪/kg
- `std_dev` - standard deviation
- `price_range_low` / `price_range_high` - min and max observed prices
- `count` - total observation count
- `count_by_source` - JSON: `{source_id: count}` breakdown
- `staleness_level` - `ok` / `warning` / `stale`
- `qa_status` - `ok` / `flagged` / `blocked`
- `qa_notes` - human-readable QA flag reason if blocked

weekly_aggregates - Same structure as daily aggregates but over 7-day windows. Used for the trend-tracking in the public report.

observation_flags - Log of admin flag actions on individual observations (audit trail).

Group 5: Pipeline Operations

pipeline_runs - A record of every pipeline execution. Each run has a type (`ingestion`, `normalize`, `aggregate`, `publish`), start/end timestamps, status, and a summary JSON with counts.

log_entries - Structured log records from pipeline stages, linked to `pipeline_runs`. This is the database-level log, not the file-level log. It enables the admin UI to surface per-run diagnostics.

publish_runs - A record of every publish execution: artifacts generated, upload status, manifest path, upload timestamp.

alerts - In-app operational alerts: QA flags, staleness warnings, source failures, admin-triggered reviews.

`pipeline_state` - A single-row table tracking the current state of the pipeline (last successful run per stage, current lock status). Used by the self-gating scheduler.

The Product Catalog: 67 Canonical Products

The catalog is organized by vegetable category. Every product has a `unit_type` that determines how its price is normalized:

Category	Example Products	Count
Leafy greens	spinach, chard, kale, lettuce (multiple varieties), parsley, cilantro, dill	18
Brassicas	cabbage (green, red, savoy), cauliflower, broccoli, kohlrabi	8
Root vegetables	carrots, beets, radishes, turnips, sweet potato	8
Alliums	onions (white, red), spring onions, leeks, garlic	7
Cucurbits	zucchini (green, yellow), cucumber (multiple varieties), pumpkin	8
Nightshades	tomatoes (multiple varieties), peppers (multiple varieties), eggplant	9
Legumes	green beans, snow peas, fava beans	3
Miscellaneous	corn, celery, fennel, artichoke, kohlrabi	5
CSA Baskets	small basket, medium basket, large basket	3

62 of 67 products currently have at least one active observation. The remaining 5 are in the catalog but have not yet been seen on any active source.

Basket Products: A Special Data Type

CSA (Community Supported Agriculture) baskets are treated differently from individual vegetable products. A basket is a weekly box that contains an assortment of seasonal vegetables – the exact contents vary by farm and by season.

Basket prices are **not decomposed to per-kg** in V1. The basket is treated as an independent product: “a small CSA basket costs ₪X per week.” This is a deliberate product decision documented in the normalization specification.

The reason: decomposing a basket into per-kg prices for individual vegetables requires knowing the exact contents and quantities – which is not consistently published. Attempting decomposition would introduce more noise than signal. Baskets as independent products give consumers a meaningful price reference point (weekly subscription cost) without false precision.

The Alias Coverage Problem

232 aliases sounds like a lot. In practice, it is a thin coverage layer over a very large problem space. Israeli farm websites exhibit extreme naming variability:

Raw Name Examples	Maps To
כרוב ירוק	cabbage (green)
כרוב קטן	cabbage (green)
כרוב ירוק קטן בשלפוחית	cabbage (green)
כרוב גינה אורגני	cabbage (green)
כרוב	cabbage (green)
קייל תלתל	kale
קייל עלים	kale
קייל מתולתל אורגני	kale

Each of these is a distinct alias entry. The alias system catches them all, but coverage must be built incrementally – every time a new source is added, or an existing source changes its product descriptions, new aliases may be needed.

The admin review workflow is designed for this: unresolvable items appear in the admin queue with the raw name, source, and price, and the operator either adds an alias (if it's a known product) or adds a scope-skip rule (if it's out of scope).

Financial Precision: No FLOAT Anywhere

A key design invariant enforced from M1: all price columns are `NUMERIC(12,4)` – no FLOAT columns in the schema. This prevents floating-point drift in price calculations.

This constraint is enforced by the M1 test suite: `tests/test_db_health.py` includes a check that queries the PostgreSQL information schema and fails if any `real` or `double precision` column is found in price-related tables.

For a price index that publishes average prices and standard deviations, floating-point drift would be invisible to users but would accumulate as the dataset grows. `NUMERIC` arithmetic is exact.